



LESSON 11: FUNCTIONS CONTINUED

Parameters, Returns & Scope

Module: Code Organization & Abstraction

Neon City Cyber Academy

Junior Agent Training Protocol

Your Team Today

Emily Chen - Handler

"Parameters let you customize functions. Return values let functions give you results. Together, they make functions incredibly powerful."

Cole Martinez - Tech Lead

"Think of parameters as INPUT and return values as OUTPUT. Functions become data transformers. That's when the real magic happens."



Quick Review: Basic Functions

Last lesson:

-  `def function_name():` to define
-  `function_name()` to call
-  DRY principle
-  Functions organize code

Today: Make functions flexible and useful!



The Problem with Basic Functions

```
def greet():  
    print("Hello, Alice!")
```

Problem: This only greets Alice. What about Bob? Charlie?

Need: A way to tell the function WHO to greet!

Solution: Parameters



Parameters: Function Inputs

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Alice") # Output: Hello, Alice!  
greet("Bob")   # Output: Hello, Bob!  
greet("Charlie") # Output: Hello, Charlie!
```

name is a **parameter** - a variable that receives data!



Emily Explains: Parameters vs Arguments

"People get confused by these terms:

Parameter = Variable in the function definition

Argument = Actual value you pass when calling

```
def greet(name): # 'name' is parameter
    print(f"Hi {name}")
```

```
greet("Alice") # 'Alice' is argument
```

Most people say 'parameters' for both - that's fine!"

Multiple Parameters

```
def create_agent(name, rank, sector):  
    print(f"Agent: {name}")  
    print(f"Rank: {rank}")  
    print(f"Sector: {sector}")  
  
create_agent("Nova", "Elite", "Downtown")
```

Output:

```
Agent: Nova  
Rank: Elite  
Sector: Downtown
```

Order matters! Arguments match parameters by position.

12
34

Parameters with Math

```
def add_numbers(a, b):  
    result = a + b  
    print(f"{a} + {b} = {result}")  
  
add_numbers(5, 3)    # Output: 5 + 3 = 8  
add_numbers(10, 20) # Output: 10 + 20 = 30
```

Now the function is reusable for ANY numbers!



Cole's Power Tip: Return Values

"Printing is useful, but **returning** values is more powerful:

```
def add(a, b):  
    return a + b # Give back the result  
  
total = add(5, 3) # Store it!  
doubled = add(5, 3) * 2 # Use it in calculations!
```

Return lets you USE the result anywhere!"



The `return` Statement

```
def multiply(x, y):  
    return x * y # Send back the result  
  
result = multiply(4, 7)  
print(result) # Output: 28
```

Key difference:

- `print()` - shows on screen
- `return` - gives value back to caller



Example: Calculator Functions

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
# Use them!  
x = add(10, 5)      # x = 15  
y = multiply(x, 2)  # y = 30  
z = subtract(y, 10) # z = 20  
print(z)           # Output: 20
```



Emily's Return Rules

"Important return facts:

1. **Exits immediately** - code after return doesn't run
2. **Can return any type** - numbers, strings, booleans
3. **Only one value** - but it can be a list/tuple!
4. **No return?** - function returns **None** by default

```
def check():  
    return True  
    print("\nNever runs!\n") # Unreachable code  
"""
```

Challenge #1: Temperature Converter

Mission: Create a function that converts Celsius to Fahrenheit.

Formula: $F = C \times 9/5 + 32$

```
def celsius_to_fahrenheit(celsius):  
    # Your code here  
  
# Test it  
print(celsius_to_fahrenheit(0))    # Should be 32  
print(celsius_to_fahrenheit(100)) # Should be 212
```



Solution: Temperature Converter

```
def celsius_to_fahrenheit(celsius):  
    fahrenheit = celsius * 9/5 + 32  
    return fahrenheit  
  
# Test it  
print(celsius_to_fahrenheit(0))      # Output: 32.0  
print(celsius_to_fahrenheit(100))    # Output: 212.0  
print(celsius_to_fahrenheit(37))     # Output: 98.6
```



Functions Calling Functions with Returns

```
def square(n):  
    return n * n  
  
def sum_of_squares(a, b):  
    return square(a) + square(b)  
  
result = sum_of_squares(3, 4)  
print(result)  # Output: 9 + 16 = 25
```

Composability! Build complex functions from simple ones.



Scope: Local vs Global

Scope = where a variable can be accessed.

```
x = 10 # Global variable

def my_function():
    y = 5 # Local variable
    print(x) # ✓ Can read global
    print(y) # ✓ Can read local

my_function()
print(x) # ✓ Works
print(y) # ✗ Error! y doesn't exist here
```


Cole's Scope Explanation

"Think of scope like security clearance:

Global = everyone can see it

Local = only the function can see it

When a function ends, its local variables are **destroyed**.

They lived in that stack frame we talked about!

This is actually a feature - it prevents functions from interfering with each other!"

Modifying Global Variables

```
count = 0 # Global

def increment():
    global count # Tell Python we want the global one
    count = count + 1

increment()
increment()
print(count) # Output: 2
```

global keyword - needed to MODIFY global variables.



Example: Game Score System

```
score = 0

def add_points(points):
    global score
    score = score + points
    print(f"Score: {score}")

add_points(10)    # Score: 10
add_points(25)    # Score: 35
add_points(5)     # Score: 40
```

But... there's a better way (return the new score)!



Emily's Best Practice: Avoid Global

"Global variables are convenient but cause problems:

- Hard to track who's changing them
- Functions become dependent on external state
- Testing becomes difficult

Better approach: Use parameters and returns:

```
def add_points(current_score, points):  
    return current_score + points  
  
score = add_points(score, 10) # More explicit!  
```
```

## Challenge #2: Circle Calculator

**Mission:** Create a function that calculates circle area.

**Formula:**  $\text{area} = \pi \times r^2$  (use 3.14159 for  $\pi$ )

```
def circle_area(radius):
 # Your code here

Test
print(circle_area(1)) # ~3.14
print(circle_area(5)) # ~78.54
print(circle_area(10)) # ~314.16
```



## Solution: Circle Calculator

```
def circle_area(radius):
 pi = 3.14159
 area = pi * radius * radius
 return area

Test
print(circle_area(1)) # Output: 3.14159
print(circle_area(5)) # Output: 78.53975
print(circle_area(10)) # Output: 314.159
```

**Note:** `pi` is local to the function!



## Default Parameters

What if a parameter is optional?

```
def greet(name, greeting="Hello"):
 print(f"{greeting}, {name}!")

greet("Alice") # Output: Hello, Alice!
greet("Bob", "Hi") # Output: Hi, Bob!
greet("Charlie", "Greetings") # Output: Greetings, Charlie!
```

**greeting="Hello"** sets a default value!



## Example: Power Function

```
def power(base, exponent=2):
 return base ** exponent

print(power(5)) # $5^2 = 25$
print(power(5, 3)) # $5^3 = 125$
print(power(2, 10)) # $2^{10} = 1024$
```

Defaults make functions more flexible!





## Cole's Default Parameter Rules

"Default parameter rules:

1. Must come AFTER non-default parameters
2. Evaluated once when function is defined
3. Great for optional settings



```
def func(required, optional=5):
```



```
def func(optional=5, required): # Error!
```

This is a syntax requirement in Python!"

## Challenge #3: Discount Calculator

**Mission:** Calculate final price after discount.

```
def calculate_price(original, discount=0):
 # Return original price minus discount percentage
 # Example: calculate_price(100, 20) = 80
 pass # Your code here

Test
print(calculate_price(100)) # 100 (no discount)
print(calculate_price(100, 20)) # 80 (20% off)
print(calculate_price(50, 10)) # 45 (10% off)
```



## Solution: Discount Calculator

```
def calculate_price(original, discount=0):
 discount_amount = original * (discount / 100)
 final_price = original - discount_amount
 return final_price
```

# Test

```
print(calculate_price(100)) # 100.0
print(calculate_price(100, 20)) # 80.0
print(calculate_price(50, 10)) # 45.0
```



## Common Error: Forgetting Return

```
def add(a, b):
 sum = a + b
 # Forgot return!

result = add(5, 3)
print(result) # Output: None
```

Without `return`, function returns `None` !



## Common Error: Wrong Argument Count

```
def greet(name, age):
 print(f"{name} is {age}")

greet("Alice") # ✗ TypeError: missing argument
greet("Alice", 25, "Engineer") # ✗ TypeError: too many arguments
```

**Fix:** Match arguments to parameters!



## Emily's Function Design Checklist

"When creating functions:

1. ☒ Clear, descriptive name (verb phrase)
2. ☒ Each function does ONE thing
3. ☒ Use parameters for inputs
4. ☒ Use return for outputs
5. ☒ Avoid global variables when possible
6. ☒ Add comments for complex logic
7. ☒ Test with different inputs"



## Key Takeaways

- ✓ **Parameters** pass data INTO functions
- ✓ **Return** sends data OUT of functions
- ✓ **Scope** controls variable visibility
- ✓ **Local** variables exist only in functions
- ✓ **Global** variables accessible everywhere
- ✓ **Default parameters** provide optional values
- ✓ Functions should be **pure** when possible (no globals)

## Next Mission: Lesson 12

### Coming Up:

- Import system and modules
- Using Python's standard library
- `random` module for randomness
- `time` module for delays
- Creating your own modules!

Agent Status: Functions FULLY MASTERED 





## Mission Stats

**Concept:** Advanced Functions (Params & Returns)

**Difficulty:** ★★★★★

**Skills Unlocked:**

- Parameter passing
- Return values
- Variable scope
- Default parameters
- Professional function design

**Next Level:** Modules & Libraries

